Projeto Final: Sistema de Regressão Espectral com Bagging e Subagging

Levantamento e Implementação em Streamlit/Python

Wagner Oliveira de Araujo

IFG — 2026

Sumário

1 Levantamento **3**

1.1 Inventário das Funcionalidades Existentes 3

1.1.1 Funções do Sistema 3

1.1.2 Entradas Necessárias 3

1.1.3 Saídas Esperadas 4

1.2 Classificação por Prioridade 6

1.2.1 Essenciais (Must Have) 6

1.2.2 Desejáveis (Should Have) 7

1.2.3 Extras (Could Have) 7

1.3 Fluxos de Uso 7

1.3.1 Fluxo Principal 7

1.3.2 Fluxos Alternativos e Pontos Críticos 8

1.4 Requisitos Não Funcionais 8

2 Projeto Lógico **9**

3 Projeto Físico **10**

4 Implementação **11**

1 Levantamento

O levantamento de requisitos visou catalogar, classificar e priorizar as funcionalidades do sistema legado (Vaadin/Java) para orientar o desenvolvimento em Python/Streamlit. A análise partiu do mapeamento exaustivo da base de código existente, garantindo rastreabilidade e aderência às necessidades de pesquisadores e alunos de quimiometria.

1.1 Inventário das Funcionalidades Existentes

1.1.1 Funções do Sistema

Conforme detalhado no Quadro 1, o inventário categoriza as funcionalidades em grupos lógicos (Cálculos, Configuração, Visualização, Organização e Exportação), evidenciando que o sistema original focava estritamente na execução algorítmica, sem mecanismos nativos de exportação ou persistência de resultados.

Quadro 1 – Inventário das funções do sistema

Categoria	**Funcionalidade**	**Descrição Técnica**
Cálculos	RLM Simples (Baseline)	Regressão Linear Múltipla padrão sem técnicas de ensemble
Cálculos	RLM + Bagging	Ensemble com amostragem *com* reposição (bootstrap)
Cálculos	RLM + Subagging + CV	Ensemble com amostragem *sem* reposição + otimização por validação cruzada 5-Fold
Configuração	Parâmetro m (Bags)	Define número de amostras/ensembles para o método Bagging
Configuração	Parâmetro k (Subagging)	Define fração ou quantidade para amostragem no Subagging
Visualização	Métricas de Desempenho	Exibição de R^2, RMSE, MAE calculados no conjunto de teste
Visualização	Gráficos Comparativos	*Scatter plots* (previsto $\times$ real), resíduos e comparação entre modelos
Organização	Abas de Navegação	Separação lógica em 3 abas: "Configuração", "Resultados" e "Gráficos"
Exportação	Download de Resultados	Botões de download para métricas em CSV (`metricas.csv`) e log de execução em TXT (`relatorio.txt`)

Fonte: Wagner Oliveira de Araujo

1.1.2 Entradas Necessárias

Para que os algoritmos sejam executados corretamente, é necessário fornecer matrizes espectrais e vetores de referência em formato texto. Os arquivos e parâmetros exigidos estão listados no Quadro 2, que especifica a estrutura dimensional esperada para calibração e teste, além dos limites válidos para os hiperparâmetros m e k.

Quadro 2 – Entradas necessárias para execução do sistema

Tipo	Nome/Parâmetro	Descrição
Arquivo	`Xcal.txt`	Matriz de espectros de calibração (variáveis independentes)
Arquivo	`Ycal.txt`	Vetor de valores de referência para calibração
Arquivo	`Xteste.txt`	Matriz de espectros para predição/teste
Arquivo	`Yteste.txt`	Vetor de valores reais para validação do conjunto de teste
Parâmetro	m (nº de bags)	Lista de inteiros ≥ 1 separados por vírgula. Ex.: 10, 20, 30, 40, 50
Parâmetro	k (Subagging)	Lista de decimais em $(0, 1]$ separados por vírgula. Ex.: 0.10, 0.20, 0.30, 0.40, 0.50
Config.	Validação Cruzada	Fixo: 5-Fold. Parâmetro interno de otimização do modelo

Fonte: Wagner Oliveira de Araujo

1.1.3 Saídas Esperadas

Após o processamento dos dados de entrada, o sistema gera indicadores quantitativos e representações visuais para suporte à decisão técnica. As saídas geradas pelo sistema são apresentadas no Quadro 3, descrevendo os formatos de exibição adotados para métricas, gráficos comparativos, resíduos e feedbacks de interface.

Quadro 3 – Saídas esperadas do sistema

Tipo	Conteúdo	Formato de Exibição
Numérico	R², RMSE, MAE e Tempo (ms) por modelo	Cards destacados por cor + tabela comparativa interativa
Gráfico	Previsto vs. Real por modelo	Gráfico de linha (Plotly) com séries Real e Predito
Gráfico	Dispersão Real $\times$ Predito (3 modelos)	Scatter plot Plotly com linha ideal $y = x$
Comparativo	Desempenho entre os três modelos	Tabela `DataFrame` com RMSE, MAE, R² e Tempo
Exportação	Métricas tabulares	Download `metricas.csv` via `st.download_button`
Exportação	Log completo de execução	Download `relatorio.txt` via `st.download_button`
Feedback	Mensagens de erro/validação	`st.error()` com descrição orientativa
Status	Indicador de processamento	`st.progress()` com texto descritivo por etapa

Fonte: Wagner Oliveira de Araujo

Com base no inventário das funcionalidades, entradas e saídas, os requisitos funcionais do sistema foram formalizados e organizados em cinco grupos temáticos (RF01–RF05), cada um subdividido em sub-itens que descrevem comportamentos específicos e verificáveis. Essa formalização, apresentada no Quadro 4, é o ponto de partida para a priorização MoSCoW, os testes de validação e a matriz de rastreabilidade.

Quadro 4 – Especificação dos requisitos funcionais (RF01–RF05)

ID	Descrição do Requisito
RF01 — Entrada e Validação de Dados	
RF01.1	O sistema deve permitir o upload dos arquivos `Xcal.txt`, `Ycal.txt`, `Xteste.txt` e `Yteste.txt` via `st.file_uploader`
RF01.2	O sistema deve aceitar apenas arquivos com extensão `.txt`, bloqueando outros formatos no widget de upload
RF01.3	O sistema deve exibir mensagem de erro descritiva em caso de falha na leitura ou conversão dos arquivos, interrompendo a execução via `st.stop()`
RF02 — Configuração e Otimização	
RF02.1	O sistema deve permitir a entrada de múltiplos valores de m (inteiros ≥ 1) separados por vírgula via campo de texto
RF02.2	O sistema deve permitir a entrada de múltiplos valores de k (decimais em $(0, 1]$) separados por vírgula via campo de texto
RF02.3	O sistema deve executar validação cruzada 5-Fold sobre todas as combinações de m e k para selecionar automaticamente os hiperparâmetros ótimos
RF03 — Execução dos Modelos	
RF03.1	O sistema deve executar a Regressão Linear Múltipla simples via pseudoinversa (`np.linalg.pinv`) como modelo baseline
RF03.2	O sistema deve executar RLM com Bagging: amostragem com reposição (*bootstrap*) usando o melhor m^* obtido por CV
RF03.3	O sistema deve executar RLM com Subagging: amostragem sem reposição com fração k^*, usando os melhores k^* e m^* obtidos por CV
RF04 — Resultados e Visualização	
RF04.1	O sistema deve calcular e exibir RMSE, MAE, R^2 e tempo de execução em ms para cada modelo via `calc_metrics()` e `time.perf_counter()`
RF04.2	O sistema deve exibir cards de métricas por modelo com código de cor (azul/verde/roxo) e tabela comparativa `DataFrame`
RF04.3	O sistema deve gerar gráfico de linha (Plotly) com valores reais e preditos por amostra para cada modelo
RF04.4	O sistema deve gerar gráfico de dispersão Real $\times$ Predito unificado com os três modelos e linha ideal $y = x$
RF04.5	O sistema deve disponibilizar download de métricas em `metricas.csv` e do log em `relatorio.txt` via `st.download_button`
RF05 — Interface e Usabilidade	
RF05.1	O sistema deve organizar o fluxo em 3 abas: "Configuração" (upload + parâmetros), "Resultados" (métricas + log) e "Gráficos"
RF05.2	O sistema deve exibir barra de progresso (`st.progress`) com texto descritivo por etapa durante o processamento
RF05.3	O sistema deve persistir os resultados da análise na sessão via `st.session_state["result"]`, permitindo navegação entre abas sem reprocessamento

Fonte: Wagner Oliveira de Araujo

1.2 Classificação por Prioridade

A priorização foi realizada por meio da técnica MoSCoW[1], separando as funcionalidades em essenciais, desejáveis e extras. Conforme o Quadro 5, os requisitos essenciais (Must Have) garantem o funcionamento mínimo do sistema, abrangendo desde o upload e parse dos arquivos até a exibição de métricas, gráficos e downloads para tomada de decisão.

1.2.1 Essenciais (Must Have)

Quadro 5 – Requisitos essenciais do sistema (Must Have)

ID	Requisito	Implementação no app.py
E01	Upload dos 4 arquivos (`.txt`)	`st.file_uploader` com `type=["txt"]`; botão bloqueado enquanto algum arquivo estiver ausente
E02	Execução da RLM Simples (baseline)	`perform_rlm()` via `np.linalg.pinv(Xcal) @ Ycal`
E03	Cálculo de métricas (R^2, RMSE, MAE, Tempo)	`calc_metrics()` + `time.perf_counter()` para cada modelo
E04	Execução do Bagging e Subagging com CV	`perform_bagging()` + `find_optimal_parameters()` com *grid search* 5-Fold
E05	Exibição de resultados e downloads	Cards, tabela `DataFrame`, gráficos Plotly e `st.download_button`

Fonte: Wagner Oliveira de Araujo

[1] MoSCoW: acrônimo para *Must Have* (obrigatório), *Should Have* (importante), *Could Have* (desejável) e *Won't Have* (fora do escopo atual). Método amplamente adotado em engenharia de software e gestão ágil de projetos para priorização de requisitos.

Os requisitos desejáveis não bloqueiam a execução, mas agregam valor à experiência do usuário e à reprodutibilidade. O Quadro 6 lista os itens Should Have indicando quais já estão implementados no `app.py` e quais permanecem como evolução futura.

1.2.2 Desejáveis (Should Have)

Quadro 6 – Requisitos desejáveis do sistema (Should Have)

ID	Requisito	Situação
D01	Gráficos interativos (zoom, tooltip)	**Implementado** — Plotly com zoom, pan e tooltip nativos
D02	Exportação de resultados (CSV, TXT)	**Implementado** — `metricas.csv` e `relatorio.txt` via `st.download_button`
D03	Tooltips explicativos para *m* e *k*	**Implementado** — parâmetro `help=` nos campos `st.text_input()`
D04	Pré-processamento básico (normalização)	Não implementado — evolução futura
D05	Histórico de execuções na sessão	Não implementado — apenas a última execução é armazenada em `session_state`

Fonte: Wagner Oliveira de Araujo

Para fases futuras de evolução do sistema, foram mapeados requisitos que ampliam a escalabilidade e a integração com ecossistemas de dados. Conforme o Quadro 7, os itens Could Have incluem autenticação, suporte a múltiplos formatos e pipelines automatizadas, alinhando a ferramenta a demandas corporativas e acadêmicas avançadas.

1.2.3 Extras (Could Have)

Quadro 7 – Requisitos extras para fases futuras (Could Have)

ID	Requisito	Cenário de Uso
X01	Autenticação e projetos privados	Ambientes corporativos ou pesquisa com dados sensíveis
X02	Integração com APIs externas	Pipelines automatizadas de quimiometria
X03	Suporte a formatos (.csv, .xlsx, .mat)	Flexibilidade para ingestão de diferentes fontes
X04	Otimização automática de hiperparâmetros	Reduz necessidade de conhecimento técnico avançado
X05	Dashboard comparativo multi-rodada	Análise estatística de estabilidade dos ensembles

Fonte: Wagner Oliveira de Araujo

1.3 Fluxos de Uso

1.3.1 Fluxo Principal

O fluxo principal de execução segue a sequência implementada no `app.py`: **Upload** dos quatro arquivos na aba "Configuração" → **Entrada** das listas de *m* e *k* via campos de texto

→ **Execução** pelo botão "Executar Análise com Otimização CV" (habilitado apenas quando os 4 arquivos estão presentes) → **Parse** e leitura dos arquivos → **Grid search** com CV 5-Fold para seleção de hiperparâmetros ótimos → **Avaliação final** dos três modelos no conjunto de teste → **Exibição** de métricas, gráficos e downloads nas abas "Resultados" e "Gráficos".

1.3.2 Fluxos Alternativos e Pontos Críticos

A robustez do sistema depende também do tratamento proativo de falhas e da mitigação de pontos críticos de usabilidade. A matriz de exceções, apresentada no Quadro 8, estabelece comportamentos esperados para entradas inválidas, timeouts e navegação inconsistente, garantindo que o usuário seja orientado a corrigir o fluxo sem perda de contexto.

Quadro 8 – Tratamento de exceções e usabilidade

Cenário	**Comportamento do Sistema**	**Implementação no app.py**
Arquivo não selecionado	Botão de execução desabilitado (`disabled=not files_ok`)	Verificação `all([xcal, ycal, xteste, yteste]) + st.info()` orientativo
Arquivo com formato inválido	`st.error()` com mensagem + `st.stop()`	Bloco `try/except` em torno de `parse_matrix()` e `parse_vector()`
Parâmetros fora do intervalo	`st.error()` com descrição do valor inválido + `st.stop()`	Funções `parse_int_list()` e `parse_float_list()` com `raise ValueError`
Erro na análise	`st.error()` + `progress_bar.empty()`	Bloco `try/except` em torno de `run_full_analysis()`
Navegação sem executar	Mensagem informativa nas abas Resultados e Gráficos	`st.info()` condicional a `"result"not in st.session_state`

Fonte: Wagner Oliveira de Araujo

1.4 Requisitos Não Funcionais

Além dos requisitos funcionais, o sistema deve atender a restrições técnicas que garantem qualidade, segurança e desempenho. O Quadro 9 especifica os critérios observados na implementação, abrangendo limites de tempo de resposta, clareza das mensagens de erro, isolamento de sessão e comportamento sob múltiplos usuários simultâneos.

Quadro 9 – Especificação de requisitos não funcionais

Categoria	**Especificação**	**Critério de Aceite**
Performance	Tempo de cálculo (RLM)	RLM $<$ 10 s; medido via `time.perf_counter()` e exibido em ms
Performance	Tempo de cálculo (Ensemble + CV)	Ensemble $<$ 2 min para os valores padrão de m e k; barra de progresso indica avanço
Performance	Renderização de gráficos	$<$ 3 s após conclusão do cálculo; renderização delegada ao Plotly no cliente
Usabilidade	Clareza textual e recuperação de erro	Mensagens de `st.error()` descrevem o valor inválido e orientam a correção
Usabilidade	Tooltips de parâmetros	Parâmetros m e k possuem `help=` nos `st.text_input()` com exemplos de uso
Segurança	Isolamento de sessão	Dados armazenados em `st.session_state` por sessão, isolados pelo Streamlit
Escalabilidade	Crescimento de usuários/dados	Arquitetura *stateless* do Streamlit suporta múltiplas sessões simultâneas independentes

Fonte: Wagner Oliveira de Araujo

2 Projeto Lógico

A estrutura do `app.py` concentra toda a lógica em um único arquivo, com funções bem definidas e separadas por responsabilidade. O Quadro 10 sintetiza as características arquiteturais identificadas no código, destacando as decisões de implementação adotadas para parsing, cálculo, otimização e apresentação.

O fluxo lógico implementado segue: `Upload` $\rightarrow$ `Parse + validação de formato` $\rightarrow$ `Configuração de listas de` m `e` k $\rightarrow$ `Grid search com CV 5-Fold` $\rightarrow$ `Avaliação final no conjunto de teste` $\rightarrow$ `Exibição de métricas, gráficos e downloads`. Todas as funções de cálculo são independentes da interface Streamlit, recebendo e retornando arrays NumPy, o que as torna testáveis isoladamente.

Quadro 10 – Características arquiteturais do app.py

Aspecto	**Decisão de Implementação**	**Função / Trecho no app.py**
Parsing de dados	Leitura linha a linha com substituição de vírgula por ponto	`parse_matrix()`, `parse_vector()`
Cálculo RLM	Pseudoinversa NumPy: `pinv(X) @ Y`	`perform_rlm()`
Bagging/Subagging	Função unificada com flag `with_replacement` e `seed` configurável	`perform_bagging()`
Validação cruzada	5-Fold com permutação aleatória; retorna média e desvio do RMSE	`run_cross_validation()`
Otimização	Busca exaustiva em grade (*grid search*) sobre listas de m e k	`find_optimal_parameters()`
Métricas	RMSE, MAE e R^2 calculados vetorialmente; tempo via `perf_counter`	`calc_metrics()`
Persistência	Resultado armazenado em `st.session_state["result"]` após execução	`run_full_analysis()`
Visualização	Gráficos interativos Plotly: linhas e dispersão com tooltip	`build_chart()`, aba Gráficos
Exportação	`st.download_button` para `metricas.csv` e `relatorio.txt`	Aba Resultados

Fonte: Wagner Oliveira de Araujo

3 Projeto Físico

O projeto é implementado em um único arquivo `app.py`, organizado em blocos funcionais delimitados por comentários. Essa estrutura concentra apresentação e lógica de negócio no mesmo arquivo, com separação clara por funções nomeadas. O Quadro 11 descreve cada bloco funcional e sua responsabilidade.

O ambiente de execução requer Python 3.10+, Streamlit, NumPy, Pandas e Plotly. Não há dependências externas além dessas bibliotecas, e o sistema não utiliza banco de dados nem sistema de arquivos persistente — todos os dados ficam em memória durante a sessão.

Quadro 11 – Estrutura funcional do app.py

Bloco / Função	**Responsabilidade**
Configuração da página	`st.set_page_config()` + CSS personalizado que replica o visual Vaadin
`parse_matrix()`	Leitura de arquivo `.txt` como matriz NumPy (linhas × colunas)
`parse_vector()`	Leitura de arquivo `.txt` como vetor NumPy (um valor por linha)
`perform_rlm()`	RLM simples via `np.linalg.pinv(Xcal) @ Ycal`
`perform_bagging()`	Bagging e Subagging unificados com flag `with_replacement` e `seed`
`calc_metrics()`	Cálculo vetorial de RMSE, MAE e R^2
`run_cross_validation()`	Validação cruzada 5-Fold; retorna média e desvio-padrão do RMSE
`find_optimal_parameters()`	*Grid search* sobre listas de m e k; gera log textual das iterações
`run_full_analysis()`	Orquestra parsing, otimização, avaliação final e montagem do dicionário de resultado
`parse_int_list()` / `parse_float_list()`	Validação e conversão das listas de m e k digitadas pelo usuário
`metric_card_html()`	Gera HTML dos cards de métricas com cor por modelo
`build_chart()`	Cria figura Plotly com séries Real e Predito por modelo
Aba Configuração	Upload dos 4 arquivos + entradas de m e k + botão de execução
Aba Resultados	Cards, tabela comparativa, log e botões de download
Aba Gráficos	Gráficos de linha por modelo + scatter comparativo

Fonte: Wagner Oliveira de Araujo

4 Implementação

A implementação materializa o projeto lógico e físico em código Python, mantendo fidelidade matemática ao original e ampliando usabilidade e reprodutibilidade. Para garantir a qualidade do código, o Quadro 12 define a estratégia de verificação, estabelecendo metas de cobertura por tipo de teste e critérios de aceite baseados em desempenho e experiência do usuário (SUS ≥ 75).

Quadro 12 – Estratégia de testes e critérios de aceite

Tipo de Teste	**Escopo**	**Meta de Cobertura**
Unitário	Funções isoladas (parse, metrics)	$\geq 80\%$ (`pytest-cov`)
Integração	Fluxo completo: Upload $\rightarrow$ CV $\rightarrow$ Gráficos	100% dos caminhos críticos
Performance	Tempo de resposta para datasets típicos	RLM < 10 s; Ensemble < 2 min
Aceite (UAT)[a]	Validação por pesquisadores/alunos	SUS[b] ≥ 75 (Bom/Excelente)

Fonte: Wagner Oliveira de Araujo

[a] UAT (*User Acceptance Testing*): Teste de Aceitação do Usuário — fase em que usuários reais validam se o sistema atende às suas necessidades antes da entrega final.

[b] SUS (*System Usability Scale*): questionário padronizado de 10 perguntas criado por John Brooke (1986) para medir a usabilidade percebida de um sistema. O *score* varia de 0 a 100, sendo: ≥ 85 Excelente, 75–84 Bom, 68–74 Aceitável e ≤ 67 Ruim.

O trecho a seguir ilustra a implementação vetorizada da função de Bagging/Subagging, com *seed* configurável para reprodutibilidade:

```
def perform_bagging(Xcal, Ycal, Xteste, num_bags, with_replacement,
    fraction, seed=42):
    """RLM + Bagging / Subagging com vetorização e seed configurá
       vel."""
    n = Xcal.shape[0]
    sample_size = n if with_replacement else max(1, int(n *
       fraction))
    rng = np.random.default_rng(seed)
    Ypred = np.zeros(Xteste.shape[0])
    for _ in range(num_bags):
        idx = rng.integers(0, n, size=sample_size) if
           with_replacement else \
              rng.choice(n, size=sample_size, replace=False)
        beta = np.linalg.pinv(Xcal[idx]) @ Ycal[idx]
        Ypred += (Xteste @ beta) / num_bags
    return Ypred
```

A rastreabilidade completa é verificada por meio da matriz apresentada no Quadro 13, que vincula cada grupo de requisitos às fases clássicas de engenharia de software e aos artefatos de validação correspondentes. Essa matriz assegura que nenhuma funcionalidade levantada seja perdida durante a transição de requisitos para código, permitindo auditoria técnica e controle de escopo em todo o ciclo de desenvolvimento.

Quadro 13 – Matriz de rastreabilidade de requisitos (*Requirements Traceability Matrix* - RTM)

ID Requisito	**Fase Clássica**	**Artefato de Validação**
RF01.1–01.3	1, 3, 5	`st.file_uploader`, `parse_matrix()`, `parse_vector()`, `st.error()` + `st.stop()`
RF02.1–02.3	1, 2, 4	`parse_int_list()`, `parse_float_list()`, `run_cross_validation()`, `find_optimal_parameters()`
RF03.1–03.3	2, 3, 4	`perform_rlm()`, `perform_bagging()`, `run_full_analysis()`
RF04.1–04.5	2, 4, 5	`calc_metrics()`, `metric_card_html()`, `build_chart()`, `st.download_button`
RF05.1–05.3	1, 3, 5	`st.tabs()`, `st.progress()`, `st.session_state["result"]`, `st.info()`

Fonte: Wagner Oliveira de Araujo

A implementação resultante atende aos requisitos funcionais e não funcionais levantados, reduz a curva de aprendizado para usuários não especialistas e estabelece base técnica sólida para evoluções futuras (autenticação, pipelines automatizados, suporte a novos formatos). A integração explícita entre texto e quadros, conforme demonstrado ao longo deste documento, garante conformidade com as normas acadêmicas e rastreabilidade técnica completa.